DEEPSEEK

1. Stack-Based Question: Daily Rainfall Tracker

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct {
    int data[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

int isFull(Stack *s) {
    return s->top == MAX - 1;
}

void push(Stack *s, int value) {
    if (!isFull(s)) {
        s->data[++(s->top)] = value;
    }
}

int pop(Stack *s) {
    if (!isEmpty(s)) {
        return s->data[(s->top)--];
    }
    return -1;
}

int main() {
```

```
    int n, rainfall, count = 0;
    Stack s;
    initStack(&s);

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &rainfall);
        push(&s, rainfall);
        if (rainfall > 50) count++;
    }

    while (!isEmpty(&s)) {
        printf("%d ", pop(&s));
    }
    printf("\n%d\n", count);

    return 0;
}
```

2. Linear Queue-Based Question: Bus Arrival Scheduler

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct {
    int data[MAX];
    int front, rear;
} Queue;

void initQueue(Queue *q) {
    q->front = q->rear = -1;
}

int isEmpty(Queue *q) {
    return q->front == -1;
```

```
}

int isFull(Queue *q) {
    return q->rear == MAX - 1;
}

void enqueue(Queue *q, int value) {
    if (isEmpty(q)) {
        q->front = q->rear = 0;
    } else {
        q->rear++;
    }
    q->data[q->rear] = value;
}

int dequeue(Queue *q) {
    int value = q->data[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    } else {
        q->front++;
    }
    return value;
}

int main() {
    int n, time, count = 0;
    Queue q;
    initQueue(&q);

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &time);
        enqueue(&q, time);
        if (time < 480) count++;
    }

    while (!isEmpty(&q)) {
        printf("%d ", dequeue(&q));
```

```
    }
    printf("\n%d\n", count);

    return 0;
}
```

3. Circular Queue-Based Question: Temperature Sensor Buffer

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    int front, rear;
    int size, capacity;
} CircularQueue;

void initQueue(CircularQueue *q, int capacity) {
    q->data = (int*)malloc(capacity * sizeof(int));
    q->front = q->rear = -1;
    q->size = 0;
    q->capacity = capacity;
}

int isEmpty(CircularQueue *q) {
    return q->size == 0;
}

int isFull(CircularQueue *q) {
    return q->size == q->capacity;
}

void enqueue(CircularQueue *q, int value) {
    if (isEmpty(q)) {
        q->front = q->rear = 0;
    } else {
        q->rear = (q->rear + 1) % q->capacity;
    }
```

```
    q->data[q->rear] = value;

    if (isFull(q)) {
        q->front = (q->front + 1) % q->capacity;
    } else {
        q->size++;
    }
}

int main() {
    int n, k, temp, count = 0;
    scanf("%d %d", &n, &k);

    CircularQueue q;
    initQueue(&q, k);

    for (int i = 0; i < n; i++) {
        scanf("%d", &temp);
        enqueue(&q, temp);
    }

    // Print from oldest to newest
    for (int i = 0; i < q.size; i++) {
        int index = (q.front + i) % q.capacity;
        printf("%d ", q.data[index]);
        if (q.data[index] > 30) count++;
    }
    printf("\n%d\n", count);

    free(q.data);
    return 0;
}
```

4. Singly Linked List Operations with Integers

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertFirst(Node **head, int data) {
    Node *newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}

void insertAfter(Node *head, int y, int x) {
    Node *current = head;
    while (current != NULL && current->data != y) {
        current = current->next;
    }
    if (current != NULL) {
        Node *newNode = createNode(x);
        newNode->next = current->next;
        current->next = newNode;
    }
}

void insertBefore(Node **head, int y, int x) {
    if (*head == NULL) return;

    if ((*head)->data == y) {
        insertFirst(head, x);
        return;
    }

    Node *current = *head;
```

```
    while (current->next != NULL && current->next->data != y) {
        current = current->next;
    }
    if (current->next != NULL) {
        Node *newNode = createNode(x);
        newNode->next = current->next;
        current->next = newNode;
    }
}

void deleteNode(Node **head, int x) {
    if (*head == NULL) return;

    if ((*head)->data == x) {
        Node *temp = *head;
        *head = (*head)->next;
        free(temp);
        return;
    }

    Node *current = *head;
    while (current->next != NULL && current->next->data != x) {
        current = current->next;
    }
    if (current->next != NULL) {
        Node *temp = current->next;
        current->next = current->next->next;
        free(temp);
    }
}

void printList(Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
```

```
    Node *head = NULL;
    int n;
    char operation[20];
    int x, y;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", operation);

        if (strcmp(operation, "insertFirst") == 0) {
            scanf("%d", &x);
            insertFirst(&head, x);
        } else if (strcmp(operation, "insertAfter") == 0) {
            scanf("%d %d", &y, &x);
            insertAfter(head, y, x);
        } else if (strcmp(operation, "insertBefore") == 0) {
            scanf("%d %d", &y, &x);
            insertBefore(&head, y, x);
        } else if (strcmp(operation, "delete") == 0) {
            scanf("%d", &x);
            deleteNode(&head, x);
        }
    }

    printList(head);

    // Free memory
    while (head != NULL) {
        Node *temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}
```

5. Circular Linked List: Music Playlist Rotator

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = newNode; // Point to itself for circular
    return newNode;
}

void addSong(Node **head, int data) {
    Node *newNode = createNode(data);

    if (*head == NULL) {
        *head = newNode;
    } else {
        Node *temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = *head;
    }
}

void deleteSong(Node **head, int data) {
    if (*head == NULL) return;

    Node *current = *head, *prev = NULL;

    // Find the node to delete
    do {
        if (current->data == data) break;
```

```
        prev = current;
        current = current->next;
    } while (current != *head);

    if (current->data != data) return; // Not found

    // If only one node
    if (current->next == current) {
        free(current);
        *head = NULL;
        return;
    }

    // If deleting head
    if (current == *head) {
        // Find last node
        Node *last = *head;
        while (last->next != *head) {
            last = last->next;
        }
        *head = current->next;
        last->next = *head;
    } else {
        prev->next = current->next;
    }

    free(current);
}

void playNext(Node **head) {
    if (*head != NULL) {
        *head = (*head)->next;
    }
}

void showPlaylist(Node *head) {
    if (head == NULL) return;

    Node *temp = head;
    do {
```

```
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("\n");
}

int main() {
    Node *head = NULL;
    int n;
    char operation[20];
    int x;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", operation);

        if (strcmp(operation, "addSong") == 0) {
            scanf("%d", &x);
            addSong(&head, x);
        } else if (strcmp(operation, "deleteSong") == 0) {
            scanf("%d", &x);
            deleteSong(&head, x);
        } else if (strcmp(operation, "playNext") == 0) {
            playNext(&head);
        } else if (strcmp(operation, "showPlaylist") == 0) {
            showPlaylist(head);
        }
    }

    // Free memory
    if (head != NULL) {
        Node *current = head;
        Node *next;
        do {
            next = current->next;
            free(current);
            current = next;
        } while (current != head);
    }
```

```
    return 0;
}
```

6. Doubly Linked List: Browser History Navigation

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = newNode->next = NULL;
    return newNode;
}

void visit(Node **current, int data) {
    // Clear forward history
    if (*current != NULL) {
        Node *temp = (*current)->next;
        while (temp != NULL) {
            Node *toDelete = temp;
            temp = temp->next;
            free(toDelete);
        }
        (*current)->next = NULL;
    }

    Node *newNode = createNode(data);
    if (*current == NULL) {
        *current = newNode;
```

```
    } else {
        (*current)->next = newNode;
        newNode->prev = *current;
        *current = newNode;
    }
}

int back(Node **current) {
    if (*current != NULL && (*current)->prev != NULL) {
        *current = (*current)->prev;
        return 1;
    }
    return 0;
}

int forward(Node **current) {
    if (*current != NULL && (*current)->next != NULL) {
        *current = (*current)->next;
        return 1;
    }
    return 0;
}

void history(Node *current) {
    // Find the first node
    Node *first = current;
    while (first != NULL && first->prev != NULL) {
        first = first->prev;
    }

    // Print from first to last
    while (first != NULL) {
        printf("%d ", first->data);
        first = first->next;
    }
    printf("\n");
}

int main() {
    Node *current = NULL;
```

```
    int n;
    char operation[10];
    int x;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", operation);

        if (strcmp(operation, "visit") == 0) {
            scanf("%d", &x);
            visit(&current, x);
        } else if (strcmp(operation, "back") == 0) {
            back(&current);
        } else if (strcmp(operation, "forward") == 0) {
            forward(&current);
        } else if (strcmp(operation, "history") == 0) {
            history(current);
        }
    }

    // Free memory
    if (current != NULL) {
        // Go to first node
        Node *first = current;
        while (first->prev != NULL) {
            first = first->prev;
        }

        // Free all nodes
        while (first != NULL) {
            Node *temp = first;
            first = first->next;
            free(temp);
        }
    }

    return 0;
}
```

7. Student Score Organizer (BST)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

Node* insert(Node *root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    } else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    return root;
}

void inorder(Node *root, int *count) {
    if (root != NULL) {
        inorder(root->left, count);
        printf("%d ", root->data);
        if (root->data > 75) (*count)++;
        inorder(root->right, count);
```

```
    }
}

void freeTree(Node *root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main() {
    Node *root = NULL;
    int n, score, count = 0;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &score);
        root = insert(root, score);
    }

    inorder(root, &count);
    printf("\n%d\n", count);

    freeTree(root);
    return 0;
}
```

8. Product Price Tracker (BST)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
```

```
} Node;

Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

Node* insert(Node *root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    } else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    return root;
}

int search(Node *root, int data) {
    if (root == NULL) return 0;

    if (root->data == data) return 1;

    if (data < root->data) {
        return search(root->left, data);
    } else {
        return search(root->right, data);
    }
}

void reverseInorder(Node *root) {
    if (root != NULL) {
        reverseInorder(root->right);
        printf("%d ", root->data);
        reverseInorder(root->left);
```

```
    }
}

void freeTree(Node *root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main() {
    Node *root = NULL;
    int n, price, p;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &price);
        root = insert(root, price);
    }

    scanf("%d", &p);

    reverseInorder(root);
    printf("\n");

    if (search(root, p)) {
        printf("Found\n");
    } else {
        printf("Not Found\n");
    }

    freeTree(root);
    return 0;
}
```

9. Library Book ID Manager (BST)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node* createNode(int data) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

Node* insert(Node *root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    } else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    return root;
}

Node* minValueNode(Node *node) {
    Node *current = node;
    while (current && current->left != NULL) {
        current = current->left;
    }
    return current;
}

Node* deleteNode(Node *root, int data) {
```

```
    if (root == NULL) return root;

    if (data < root->data) {
        root->left = deleteNode(root->left, data);
    } else if (data > root->data) {
        root->right = deleteNode(root->right, data);
    } else {
        if (root->left == NULL) {
            Node *temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node *temp = root->left;
            free(root);
            return temp;
        }

        Node *temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void inorder(Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void freeTree(Node *root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}
```

```c
int main() {
    Node *root = NULL;
    int n, id, d;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &id);
        root = insert(root, id);
    }

    scanf("%d", &d);
    root = deleteNode(root, d);

    inorder(root);
    printf("\n");

    freeTree(root);
    return 0;
}
```

10. Bubble Sort: Temperature Stabilizer

```c
#include <stdio.h>

void bubbleSort(int arr[], int n, int *swapCount) {
    int i, j, temp;
    *swapCount = 0;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                (*swapCount)++;
            }
```

```
        }
    }
}

int main() {
    int n, swapCount;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    bubbleSort(arr, n, &swapCount);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n%d\n", swapCount);

    return 0;
}
```

11. Insertion Sort: Student Roll Organizer

```c
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, j, key;

    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
```

```
        arr[j + 1] = key;

        // Print after each insertion
        for (int k = 0; k <= i; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n");
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Print first element
    printf("%d\n", arr[0]);

    insertionSort(arr, n);

    return 0;
}
```

12. Merge Sort: Lab Sample Sorter

```c
#include <stdio.h>
#include <stdlib.h>

int mergeCount = 0;

void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
```

```
    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    mergeCount++;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    mergeSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n%d\n", mergeCount);

    return 0;
}
```

13. Selection Sort: Price Tag Organizer

```c
#include <stdio.h>

void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;

    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
```

```
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }

        // Swap
        temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;

        // Print after each pass
        for (int k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n");
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    selectionSort(arr, n);

    return 0;
}
```

14. Quick Sort: Delivery Time Optimizer

```c
#include <stdio.h>

void swap(int* a, int* b) {
    int temp = *a;
```

```
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
```

```
    }
    printf("\n");

    return 0;
}
```

15. Radix Sort: Invoice Number Sorter

```c
#include <stdio.h>

int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

void countSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    for (int i = 0; i < n; i++) {
        count[(arr[i] / exp) % 10]++;
    }

    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }

    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    for (int i = 0; i < n; i++) {
```

```
        arr[i] = output[i];
    }
}

void radixSort(int arr[], int n) {
    int max = getMax(arr, n);

    for (int exp = 1; max / exp > 0; exp *= 10) {
        countSort(arr, n, exp);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    radixSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

16. Binary Search: Book Finder

```c
#include <stdio.h>

int binarySearch(int arr[], int left, int right, int x) {
    while (left <= right) {
        int mid = left + (right - left) / 2;
```

```
        if (arr[mid] == x) {
            return 1;
        }

        if (arr[mid] < x) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return 0;
}

int main() {
    int n, x;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);

    if (binarySearch(arr, 0, n - 1, x)) {
        printf("Found\n");
    } else {
        printf("Not Found\n");
    }

    return 0;
}
```

17. Linear Probing: Student Hash Table

```c
#include <stdio.h>
```

```c
#include <stdlib.h>

void insertLinearProbing(int hashTable[], int m, int key) {
    int index = key % m;
    int i = 0;

    while (hashTable[(index + i) % m] != -1) {
        i++;
    }

    hashTable[(index + i) % m] = key;
}

int main() {
    int n, m;
    scanf("%d", &n);

    int rollNumbers[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &rollNumbers[i]);
    }

    scanf("%d", &m);

    int hashTable[m];
    for (int i = 0; i < m; i++) {
        hashTable[i] = -1;
    }

    for (int i = 0; i < n; i++) {
        insertLinearProbing(hashTable, m, rollNumbers[i]);
    }

    printf("[");
    for (int i = 0; i < m; i++) {
        printf("%d", hashTable[i]);
        if (i < m - 1) {
            printf(", ");
        }
    }
```

```
    printf("]\n");

    return 0;
}
```

18. Quadratic Probing: Product ID Hashing

```c
#include <stdio.h>
#include <stdlib.h>

void insertQuadraticProbing(int hashTable[], int m, int key) {
    int index = key % m;
    int i = 0;

    while (hashTable[(index + i * i) % m] != -1) {
        i++;
        if (i >= m) break; // Avoid infinite loop
    }

    if (i < m) {
        hashTable[(index + i * i) % m] = key;
    }
}

int main() {
    int n, m;
    scanf("%d", &n);

    int productIDs[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &productIDs[i]);
    }

    scanf("%d", &m);

    int hashTable[m];
    for (int i = 0; i < m; i++) {
        hashTable[i] = -1;
```

```
    }

    for (int i = 0; i < n; i++) {
        insertQuadraticProbing(hashTable, m, productIDs[i]);
    }

    printf("[");
    for (int i = 0; i < m; i++) {
        printf("%d", hashTable[i]);
        if (i < m - 1) {
            printf(", ");
        }
    }
    printf("]\n");

    return 0;
}
```

19. Tower of Hanoi: Disk Mover

```c
#include <stdio.h>

void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 1) {
        printf("Move disk 1 from rod %c to rod %c\n", from_rod, to_rod);
        return;
    }

    towerOfHanoi(n - 1, from_rod, aux_rod, to_rod);
    printf("Move disk %d from rod %c to rod %c\n", n, from_rod, to_rod);
    towerOfHanoi(n - 1, aux_rod, to_rod, from_rod);
}

int main() {
    int n;
    scanf("%d", &n);

    towerOfHanoi(n, 'A', 'C', 'B');
```

```
    return 0;
}
```

All solutions are written in C language and follow DSA best practices. They handle the exact input/output formats specified in the problems and include proper memory management where needed.